LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark DataFrame

Module 01 — Expert Class Notes

Creating DataFrames | Schema & Types | Inspecting Data | Select & Filter | Transformations | Aggregations | Joins | Null Handling | Writing Data

Expert Depth	Interview Ready	Code + Output	Analogy First
Every concept explained from first principles	Questions from FAANG, TCS, Infosys, Wipro	Every method shown with actual results	Concepts mapped to real Indian businesses

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years of industry experience, Sateesh is the founder of LevelUp Cloud Trainings — India's premier data engineering education platform. He has trained 5,000+ students in PySpark, Databricks, Microsoft Fabric, Azure, GCP and AWS.

This DataFrame 01 expert notes document contains EVERYTHING you need to not just understand PySpark DataFrames — but to master them, crack interviews, and write production-grade code.

Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: What is a DataFrame? — The Foundation

A DataFrame is the CORE data structure in PySpark. Before writing a single line of code, you must understand what a DataFrame is at a conceptual level. This is what 90% of students skip — and why they struggle later.

> **Sateesh Says 💡**
>
> *Think of a DataFrame like a massive Excel sheet — rows and columns, structured data.*
> *But unlike Excel, which runs on your laptop with 16 GB RAM, a DataFrame is DISTRIBUTED across 100s of machines.*
> *Flipkart's 10 TB orders table? In PySpark that's ONE DataFrame — split into hundreds of partitions across the cluster.*
> *You write code as if it's one table. Spark handles the distribution transparently. That's the magic!*

1.1 DataFrame vs RDD vs Dataset — Critical Comparison

Every PySpark interview will test this. Know the exact differences, when each was introduced, and why DataFrames are preferred.

Feature	RDD (Spark 1.0)	DataFrame (Spark 1.3)	Dataset (Spark 1.6)
Structure	No schema (raw objects)	Named columns + schema	Typed + named columns
Language	Python/Scala/Java/R	Python/Scala/Java/R	Scala and Java ONLY
Optimization	No Catalyst optimizer	Catalyst + Tungsten	Catalyst + Tungsten
Type Safety	Runtime errors only	Runtime errors only	Compile-time errors
Performance	Baseline (slowest)	Faster (Catalyst opts)	Same as DataFrame
API Level	Low-level (verbose)	High-level (concise)	High-level (typed)
Null Safety	Manual handling	Built-in null support	Strong type system
Use Case Today	Legacy code, custom ops	99% of modern Spark	Java/Scala projects

1.2 DataFrame Internal Architecture — What Happens Under the Hood

How a DataFrame is stored internally

```
A DataFrame = Schema + RDD of InternalRow objects

orders_df (10 GB file, 3 Executors, 6 partitions):

Schema:  order_id: INT | city: STRING | amount: DOUBLE

Executor 1 (Machine A):
  Partition 0: InternalRow(1001,'Mumbai',350.0) ...
  Partition 1: InternalRow(1002,'Delhi',200.0) ...

Executor 2 (Machine B):
  Partition 2: InternalRow(1003,'Hyd',450.0) ...
  Partition 3: InternalRow(1004,'Chennai',300.0) ...

Executor 3 (Machine C):
  Partition 4: InternalRow(1005,'Pune',180.0) ...
  Partition 5: InternalRow(1006,'Kolkata',520.0) ...

Driver sees ONE logical table, not 6 partitions.
All operations run in parallel on all partitions.
```

1.3 Catalyst Optimizer — Why DataFrames Beat RDDs on Performance

The Catalyst Optimizer is the secret weapon behind PySpark DataFrames. It automatically rewrites your queries for maximum performance — even if you write inefficient code.

Catalyst Optimizer Pipeline

```
Your Code:
df.filter(df.city=='Mumbai').select('order_id','amount')

STEP 1 — Analysis (Resolve names and types)
Checks: does column 'city' exist? Is 'order_id' valid?

STEP 2 — Logical Optimization
Rule: Predicate Pushdown -> push filter BEFORE select
Rule: Column Pruning     -> only read order_id, amount
Result: Read only 2 columns, filter before loading rest

STEP 3 — Physical Planning
Generates multiple physical plans, picks cheapest one
Chooses: broadcast hash join or sort merge join?

STEP 4 — Code Generation (Tungsten)
Generates JVM bytecode. Whole-stage code generation.
Eliminates virtual function calls. Blazing fast!
```

Key Fact: Predicate Pushdown

When you write df.filter(...).select(...), Catalyst REORDERS it to select first, then filter. Why? Reading fewer columns from disk = less I/O = faster. Spark pushes filters down to

storage layer.

With Parquet format: Spark skips entire row groups that don't match the filter. Zero bytes read!

This is why DataFrame code is often faster than handwritten RDD code, even for experienced developers.

Module 2: Creating DataFrames — All 6 Methods

Before you can process data, you need to CREATE a DataFrame. There are 6 main ways to create DataFrames in PySpark. Knowing all 6 — and when to use each — is essential for interviews and real projects.

> **Sateesh Says 💡**
>
> *Students always ask: 'Which method should I use?'*
>
> *Answer: In production, you ALWAYS read from files (CSV, Parquet, Delta). createDataFrame() is for testing only.*
>
> *In interviews, they test if you know the schema options. Master inferSchema vs explicit schema — that's the key question!*

2.1 Method 1 — createDataFrame() from Python List

Used for quick testing and prototyping. You provide data as a Python list of tuples or dictionaries.

Create DataFrame from list — 3 variations

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField
from pyspark.sql.types import StringType, IntegerType, DoubleType

spark = SparkSession.builder.appName('LevelUp').getOrCreate()

# Variation 1: List of tuples, column names as second argument
data = [('Raj', 'Mumbai', 85000),
        ('Priya', 'Delhi', 92000),
        ('Amit', 'Hyderabad', 78000),
        ('Sneha', 'Mumbai', 95000),
        ('Vikram', 'Pune', 67000)]

df = spark.createDataFrame(data, ['name', 'city', 'salary'])
df.show()

# Variation 2: Explicit schema (PRODUCTION RECOMMENDED!)
schema = StructType([
    StructField('name',   StringType(),  nullable=True),
    StructField('city',   StringType(),  nullable=True),
    StructField('salary', IntegerType(), nullable=False)
])
df = spark.createDataFrame(data, schema)

# Variation 3: List of Row objects (Row-based approach)
from pyspark.sql import Row
rows = [Row(name='Raj', city='Mumbai', salary=85000),
        Row(name='Priya', city='Delhi', salary=92000)]
```

```
df = spark.createDataFrame(rows)
```

df.show() output

```
+------+---------+------+
|  name|     city|salary|
+------+---------+------+
|   Raj|   Mumbai| 85000|
| Priya|    Delhi| 92000|
|  Amit|Hyderabad| 78000|
| Sneha|   Mumbai| 95000|
|Vikram|     Pune| 67000|
+------+---------+------+
```

2.2 Method 2 — Read CSV File (Most Common!)

Reading CSV — All important options explained

```
# Basic read (NOT recommended for production)
df = spark.read.csv('employees.csv')
# Problem: column names become _c0, _c1... and types are all String

# Production-grade read with all options
df = spark.read \
    .option('header', 'true') \        # First row = column names
    .option('inferSchema', 'true') \   # Auto-detect data types
    .option('sep', ',') \              # Delimiter (default ,)
    .option('nullValue', 'NA') \       # Treat 'NA' as null
    .option('emptyValue', '') \        # Empty string handling
    .option('dateFormat', 'dd/MM/yyyy') \  # Custom date format
    .option('multiLine', 'true') \     # For multi-line fields
    .csv('s3://data/employees.csv')

# Even better: provide schema explicitly (fastest!)
from pyspark.sql.types import *
schema = StructType([
    StructField('emp_id',   IntegerType(), True),
    StructField('name',     StringType(),  True),
    StructField('city',     StringType(),  True),
    StructField('salary',   DoubleType(),  True),
    StructField('dept',     StringType(),  True),
    StructField('join_date',DateType(),    True),
])
df = spark.read.schema(schema).option('header','true').csv('emp.csv')
```

Common Mistake / Watch Out!

inferSchema=True reads the ENTIRE file TWICE — once to infer types, once to load data. For 100 GB files this is VERY slow.

ALWAYS provide an explicit schema in production. It's faster AND prevents wrong type inference (e.g., '001' inferred as Integer not String).

Never use spark.read.csv() without header=True unless your CSV genuinely has no header row.

2.3 Method 3 — Read JSON

Reading JSON — flat and nested

```
# Flat JSON (one record per line - JSON Lines format)
df = spark.read.json('orders.json')

# JSON with options
df = spark.read \
    .option('multiLine', 'true') \   # Standard JSON array format
    .option('mode', 'PERMISSIVE') \ # Don't fail on bad records
    .json('orders.json')

# Nested JSON — Spark creates struct columns automatically
# Example JSON record:
# {'order_id':1001, 'customer':{'name':'Raj','city':'Mumbai'},
#  'items':[{'sku':'A1','qty':2},{'sku':'B2','qty':1}]}

df.printSchema()
# root
#  |-- order_id: long
#  |-- customer: struct
#  |    |-- name: string
#  |    |-- city: string
#  |-- items: array
#  |    |-- element: struct
#  |    |    |-- sku: string
#  |    |    |-- qty: long

# Access nested fields with dot notation
df.select('customer.name', 'customer.city').show()
```

2.4 Method 4 — Read Parquet (Best Format for Big Data!)

Parquet is the PREFERRED format in production Big Data pipelines. It is columnar, compressed, and schema-embedded. Always aim to convert CSV to Parquet in your pipelines.

Reading and Writing Parquet

```
# Reading Parquet - no options needed! Schema is stored in file.
df = spark.read.parquet('s3://datalake/orders/year=2024/')

# Reading with partition pruning (reads ONLY the specified folder)
df = spark.read.parquet('s3://datalake/orders/') \
    .filter('year = 2024 AND month = 12')  # Partition pruning!

# Writing DataFrame as Parquet
df.write \
    .mode('overwrite') \           # overwrite / append / ignore
    .partitionBy('city', 'year') \ # Partition output files
    .parquet('s3://output/employees/')
```

CSV	Parquet

Text format (human readable)	Binary columnar format
No schema stored in file	Schema embedded in file
Reads all columns even if unused	Column pruning: reads only needed cols
No built-in compression	Snappy/Gzip compression built in
~1x size	~4-10x smaller than CSV
No partition pruning support	Native partition pruning support
Slow for analytics queries	Up to 10x faster for analytics

2.5 Method 5 — Read from SQL Database (JDBC)

Reading from MySQL / SQL Server via JDBC

```
# Requires JDBC driver JAR in classpath
df = spark.read \
    .format('jdbc') \
    .option('url', 'jdbc:mysql://host:3306/flipkart_db') \
    .option('dbtable', 'orders') \
    .option('user', 'spark_user') \
    .option('password', 'secret') \
    .option('numPartitions', '8') \   # Parallel read partitions
    .option('partitionColumn', 'order_id') \ # Column to split on
    .option('lowerBound', '1') \
    .option('upperBound', '10000000') \
    .load()
# numPartitions + partitionColumn = parallel reads from DB! Fast!
```

2.6 Method 6 — Read Delta Table (Databricks)

Reading Delta Tables — Databricks / Delta Lake

```
# Read by path
df = spark.read.format('delta').load('/mnt/datalake/orders')

# Read by table name (Unity Catalog / Hive Metastore)
df = spark.table('catalog.schema.orders')

# Read a specific version (time travel!)
df = spark.read.format('delta') \
    .option('versionAsOf', 5) \      # Read version 5
    .load('/mnt/datalake/orders')

# Read as of a timestamp
df = spark.read.format('delta') \
    .option('timestampAsOf','2024-01-01') \
    .load('/mnt/datalake/orders')
```

Module 3: Inspecting DataFrames — Know Your Data

Before transforming data, always INSPECT it first. This is step 1 in every data engineering and data science workflow. These methods are also tested heavily in Databricks and cloud certifications.

3.1 Schema and Structure Methods

All inspection methods — with output

```
# 1. printSchema() — shows column names, types, nullable flag
df.printSchema()
```

printSchema() output

```
root
 |-- name:   string  (nullable = true)
 |-- city:   string  (nullable = true)
 |-- salary: integer (nullable = false)
```

More inspection methods

```
# 2. schema — returns StructType object (useful in code)
print(df.schema)
# StructType([StructField('name',StringType(),True), ...])

# 3. dtypes — list of (column_name, dtype_string) tuples
print(df.dtypes)
# [('name', 'string'), ('city', 'string'), ('salary', 'int')]

# 4. columns — list of column names
print(df.columns)
# ['name', 'city', 'salary']

# 5. count() — total number of rows (ACTION! triggers computation)
print(df.count())   # 5

# 6. show() — print first N rows (default 20)
df.show()            # default: top 20 rows, truncates long strings
df.show(5)           # top 5 rows only
df.show(5, truncate=False)  # no truncation, see full values
df.show(5, truncate=50)     # truncate at 50 chars
df.show(vertical=True)      # vertical format (one row per line)

# 7. describe() — statistics (count, mean, stddev, min, max)
df.describe().show()
df.describe('salary', 'age').show()  # specific columns only

# 8. summary() — extended describe (adds 25%, 50%, 75% percentiles)
df.summary().show()

# 9. first() — returns first Row object
```

```
row = df.first()
print(row['name'])   # access by column name

# 10. head(n) — returns list of first n Row objects
rows = df.head(3)    # list of 3 Row objects (Python list)

# 11. collect() — returns ALL rows as Python list (DANGEROUS on big data!)
all_rows = df.collect()  # NEVER on millions of rows! Driver OOM!

# 12. take(n) — returns first n rows as Python list (safer than collect)
some_rows = df.take(5)

# 13. toPandas() — convert to Pandas DataFrame (small datasets only!)
pdf = df.toPandas()   # DANGER: brings all data to Driver!
```

Common Mistake / Watch Out!

collect() and toPandas() bring ALL data to the Driver node. On a 100 GB DataFrame, this will crash the Driver with OutOfMemoryError.

RULE: Only use collect() or toPandas() after you have already filtered/aggregated the data down to a small result set (< 10,000 rows).

count() is an ACTION — it scans the entire DataFrame. Expensive on large data. Cache first if you need count multiple times.

Module 4: select() — Choosing Your Columns

select() is one of the most frequently used transformations. It creates a new DataFrame with only the columns you specify. Knowing all the different ways to use select() is a strong interview signal.

> **Sateesh Says** 💡
>
> *select() is like the SELECT clause in SQL — but for distributed data.*
>
> *Every column you DON'T select is PRUNED by the Catalyst Optimizer — it won't even read it from disk!*
>
> *Flipkart has 200 columns in their orders table. You need only 5. select() saves 97.5% of I/O. That's HUGE at terabyte scale.*

4.1 All Ways to Use select()

select() — 8 different patterns

```
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Pattern 1: Column names as strings
df.select('name', 'city').show()

# Pattern 2: Column objects (df.column_name)
df.select(df.name, df.city).show()

# Pattern 3: col() function (MOST FLEXIBLE — preferred in code)
df.select(F.col('name'), F.col('salary')).show()

# Pattern 4: List unpacking — select all columns dynamically
cols = ['name', 'city', 'salary']
df.select(*cols).show()
df.select(df.columns).show()   # select ALL columns

# Pattern 5: Expressions with alias
df.select(
    F.col('name').alias('employee_name'),
    F.col('salary') * 1.10 .alias('salary_with_hike'),
    F.lit('LevelUp').alias('company')   # literal constant
).show()

# Pattern 6: selectExpr() — use SQL expressions as strings!
df.selectExpr(
    'name as employee_name',
    'salary * 1.10 as salary_with_hike',
    'upper(city) as city_upper',
    'CASE WHEN salary>80000 THEN "High" ELSE "Normal" END as band'
).show()

# Pattern 7: Select with casting
df.select(F.col('salary').cast(DoubleType()).alias('salary_d'))
```

```
# Pattern 8: Select EXCEPT certain columns
drop_cols = ['salary', 'dept']
df.select([c for c in df.columns if c not in drop_cols]).show()
```

selectExpr() is Powerful — Use It!

selectExpr() accepts any Spark SQL expression as a string. This gives you full SQL power inside Python code.

You can use CASE WHEN, string functions, date functions, arithmetic, window functions — all inside selectExpr().

Performance is identical to DataFrame API — both compile to the same physical plan via Catalyst.

Use selectExpr() when the expression would be complex and verbose to write using the column API.

Module 5: filter() and where() — Filtering Rows

filter() and where() are IDENTICAL in PySpark — they are aliases. Both select rows that match a condition. This is the most common operation in any data pipeline.

5.1 All Ways to Filter

filter() / where() — Complete reference

```
from pyspark.sql import functions as F

# 1. SQL-style string expression (most readable)
df.filter("city == 'Mumbai'").show()
df.where("salary > 80000").show()   # where() = filter()

# 2. Column expression (most flexible)
df.filter(F.col('city') == 'Mumbai').show()
df.filter(df.city == 'Mumbai').show()   # same result

# 3. Multiple conditions — AND
df.filter((F.col('city') == 'Mumbai') & (F.col('salary') > 80000))
df.filter("city = 'Mumbai' AND salary > 80000")   # SQL style

# 4. Multiple conditions — OR
df.filter((F.col('city') == 'Mumbai') | (F.col('city') == 'Delhi'))
df.filter("city IN ('Mumbai','Delhi','Hyderabad')")

# 5. NOT condition
df.filter(~(F.col('city') == 'Mumbai'))
df.filter(F.col('city') != 'Mumbai')

# 6. isin() — filter with a list of values
cities = ['Mumbai', 'Delhi', 'Hyderabad']
df.filter(F.col('city').isin(cities)).show()
df.filter(~F.col('city').isin(cities)).show()  # NOT IN cities

# 7. between() — range filter (inclusive on both ends)
df.filter(F.col('salary').between(60000, 90000)).show()

# 8. like() and rlike() — pattern matching
df.filter(F.col('name').like('R%')).show()    # starts with R
df.filter(F.col('name').like('%ar%')).show()  # contains 'ar'
df.filter(F.col('email').rlike('^[a-z]+@.*\.com$')).show() # regex

# 9. startswith() / endswith() / contains()
df.filter(F.col('name').startswith('S')).show()
df.filter(F.col('email').endswith('.com')).show()
df.filter(F.col('address').contains('Mumbai')).show()

# 10. isNull() / isNotNull() — null filtering
df.filter(F.col('salary').isNull()).show()
df.filter(F.col('salary').isNotNull()).show()
df.filter("salary IS NOT NULL").show()  # SQL style
```

Common Mistake / Watch Out!

Common mistake: using Python 'and', 'or' instead of '&', '|' for column conditions.

WRONG: df.filter(df.city == 'Mumbai' and df.salary > 80000) # Python 'and' doesn't work!

RIGHT: df.filter((df.city == 'Mumbai') & (df.salary > 80000)) # Use & with parentheses

ALSO WRONG: forgetting parentheses around each condition. '&' has higher precedence than '==='.

Module 6: withColumn() & withColumnRenamed()

withColumn() is used to ADD a new column or MODIFY an existing column. withColumnRenamed() renames a column. These are the most-used transformations in data engineering ETL pipelines.

> **Sateesh Says 💡**
> *In every ETL pipeline at Infosys, TCS, or Wipro — you receive raw data and need to transform it. Add a tax column, clean the phone number, derive age from DOB, flag high-value customers... withColumn() is your primary tool. Master every pattern here and you can handle 80% of ETL requirements!*

6.1 withColumn() — Complete Reference

withColumn() — All important patterns

```
from pyspark.sql import functions as F
from pyspark.sql.types import *

# 1. Add a new constant column (literal)
df = df.withColumn('country', F.lit('India'))
df = df.withColumn('bonus_pct', F.lit(0.15))

# 2. Derived column from arithmetic
df = df.withColumn('annual_salary', F.col('salary') * 12)
df = df.withColumn('tax_amount', F.col('salary') * 0.30)
df = df.withColumn('net_salary',
    F.col('salary') - (F.col('salary') * 0.30))

# 3. CASE WHEN — if/else logic
df = df.withColumn('salary_band',
    F.when(F.col('salary') >= 90000, 'A - Senior')
     .when(F.col('salary') >= 70000, 'B - Mid')
     .when(F.col('salary') >= 50000, 'C - Junior')
     .otherwise('D - Trainee')
)

# 4. Modify EXISTING column (overwrite with same name)
df = df.withColumn('salary', F.col('salary').cast(DoubleType()))
df = df.withColumn('name', F.upper(F.col('name')))
df = df.withColumn('city', F.trim(F.col('city')))  # remove spaces

# 5. String functions
df = df.withColumn('name_upper',   F.upper(F.col('name')))
df = df.withColumn('name_lower',   F.lower(F.col('name')))
df = df.withColumn('name_initcap', F.initcap(F.col('name')))
df = df.withColumn('name_len',     F.length(F.col('name')))
df = df.withColumn('name_trimmed', F.trim(F.col('name')))
df = df.withColumn('first_3',      F.col('name').substr(1, 3))
df = df.withColumn('concat_col',
```

```
        F.concat(F.col('name'), F.lit(' - '), F.col('city')))
df = df.withColumn('concat_ws',
        F.concat_ws(' | ', F.col('name'), F.col('city')))

# 6. Date functions
df = df.withColumn('today',       F.current_date())
df = df.withColumn('now',         F.current_timestamp())
df = df.withColumn('year',        F.year(F.col('join_date')))
df = df.withColumn('month',       F.month(F.col('join_date')))
df = df.withColumn('day',         F.dayofmonth(F.col('join_date')))
df = df.withColumn('day_of_week', F.dayofweek(F.col('join_date')))
df = df.withColumn('add_30_days', F.date_add(F.col('join_date'), 30))
df = df.withColumn('date_diff',
        F.datediff(F.current_date(), F.col('join_date')))
df = df.withColumn('months_diff',
        F.months_between(F.current_date(), F.col('join_date')))

# 7. Type casting
df = df.withColumn('salary_str', F.col('salary').cast(StringType()))
df = df.withColumn('salary_dbl', F.col('salary').cast('double'))
df = df.withColumn('salary_dbl', F.col('salary').cast(DoubleType()))

# 8. Regex extraction
df = df.withColumn('area_code',
        F.regexp_extract(F.col('phone'), r'^(\d{3})', 1))
df = df.withColumn('cleaned_phone',
        F.regexp_replace(F.col('phone'), r'[^0-9]', ''))

# 9. Conditional null handling with coalesce()
df = df.withColumn('final_salary',
        F.coalesce(F.col('salary'), F.col('default_salary'), F.lit(0)))
```

6.2 withColumnRenamed() — Renaming Columns

withColumnRenamed() and rename patterns

```
# 1. Rename a single column
df = df.withColumnRenamed('name', 'employee_name')
df = df.withColumnRenamed('salary', 'base_salary')

# 2. Chain multiple renames
df = df.withColumnRenamed('name', 'emp_name') \
       .withColumnRenamed('city', 'location') \
       .withColumnRenamed('salary', 'ctc')

# 3. Rename using toDF() — rename ALL columns at once
new_cols = ['emp_id','emp_name','location','ctc','dept','doj']
df = df.toDF(*new_cols)

# 4. Rename with select + alias (most flexible)
df = df.select(
        F.col('name').alias('emp_name'),
        F.col('city').alias('location'),
        F.col('salary').alias('ctc')
)

# 5. Rename using dictionary (Python trick)
rename_map = {'name':'emp_name','city':'location','salary':'ctc'}
for old, new in rename_map.items():
```

```
    df = df.withColumnRenamed(old, new)
```

6.3 drop() — Removing Columns

drop() — Remove one or many columns

```
# Drop a single column
df = df.drop('temp_column')

# Drop multiple columns
df = df.drop('col1', 'col2', 'col3')
df = df.drop(*['col1', 'col2', 'col3'])   # unpack list

# Drop using list comprehension (keep only selected columns)
keep = ['emp_id', 'name', 'salary']
df = df.select(keep)                       # preferred for clarity

# Safely drop (only if column exists)
if 'old_column' in df.columns:
    df = df.drop('old_column')
```

Module 7: Sorting, Distinct & Deduplication

7.1 orderBy() and sort()

Sorting DataFrames — all patterns

```
# orderBy() and sort() are IDENTICAL — sort() is an alias

# Ascending (default)
df.orderBy('salary').show()
df.sort('salary').show()          # same result

# Descending
df.orderBy(F.col('salary').desc()).show()
df.orderBy(F.desc('salary')).show()      # alternative
df.orderBy('salary', ascending=False).show()  # string col name

# Multiple columns sort
df.orderBy(F.col('city').asc(), F.col('salary').desc()).show()
df.orderBy(['city', 'salary'], ascending=[True, False]).show()

# Sort nulls first or last
df.orderBy(F.col('salary').desc_nulls_last()).show()
df.orderBy(F.col('salary').asc_nulls_first()).show()
```

7.2 distinct() and dropDuplicates()

Deduplication is a core ETL operation. Knowing the difference between distinct() and dropDuplicates() is an interview favorite.

distinct() vs dropDuplicates()

```
# distinct() — removes ALL duplicate rows (compares ALL columns)
df.distinct().show()
df.distinct().count()   # count unique rows

# dropDuplicates() — same as distinct() when no args given
df.dropDuplicates().show()

# dropDuplicates() with subset — UNIQUE on specific columns
# Keep first occurrence of each unique (city, dept) combination
df.dropDuplicates(['city', 'dept']).show()

# Real-world use: keep latest record per employee
from pyspark.sql.window import Window
w = Window.partitionBy('emp_id').orderBy(F.col('updated_at').desc())
df.withColumn('rn', F.row_number().over(w)) \
  .filter('rn = 1') \
  .drop('rn') \
  .show()
```

distinct()	dropDuplicates()
Compares ALL columns	Can compare subset of columns
df.distinct()	df.dropDuplicates(['col1','col2'])
No control over which row to keep	No control over which row to keep
Full shuffle required (expensive)	Full shuffle required (expensive)
Use when entire row must be unique	Use when combination of cols unique

Module 8: groupBy() & Aggregations

groupBy() followed by aggregate functions is the foundation of analytics. In every Big Data interview, you WILL be asked to write a groupBy aggregation. Master every pattern here.

> **Sateesh Says 💡**
>
> *Think of groupBy as Swiggy's city-wise report: 'How many orders from each city? What is the average order value?'*
>
> *In SQL it is GROUP BY. In PySpark it is df.groupBy('city').agg(...).*
>
> *The KEY difference from SQL: In PySpark, groupBy triggers a SHUFFLE. This is the most expensive operation.*
>
> *Always think: Can I avoid the shuffle with a broadcast join instead? That's the senior-level thinking!*

8.1 groupBy().agg() — All Aggregate Functions

Complete groupBy aggregation reference

```
from pyspark.sql import functions as F

# Basic groupBy with agg
df.groupBy('city').agg(
    F.count('emp_id').alias('employee_count'),
    F.sum('salary').alias('total_salary'),
    F.avg('salary').alias('avg_salary'),
    F.min('salary').alias('min_salary'),
    F.max('salary').alias('max_salary'),
    F.stddev('salary').alias('stddev_salary'),
    F.variance('salary').alias('variance_salary'),
    F.countDistinct('dept').alias('unique_depts'),
    F.collect_list('name').alias('all_names'),   # list with dups
    F.collect_set('dept').alias('unique_dept_set'),  # set no dups
    F.first('name').alias('first_employee'),
    F.last('name').alias('last_employee'),
    F.sum_distinct('salary').alias('sum_unique_salaries'),
).show()

# Group by MULTIPLE columns
df.groupBy('city', 'dept').agg(
    F.count('*').alias('count'),
    F.avg('salary').alias('avg_salary')
).orderBy('city', F.col('avg_salary').desc()).show()

# count() shorthand
df.groupBy('city').count().show()  # quicker for just counts

# Shorthand aggregation with dict (less flexible but quick)
df.groupBy('city').agg({'salary':'avg','emp_id':'count'}).show()

# Pivot — rows become columns
df.groupBy('dept') \
  .pivot('city', ['Mumbai','Delhi','Hyderabad']) \
```

```
    .agg(F.avg('salary')) \
    .show()
# Creates one column per city value with avg salary
```

groupBy city result

```
+---------+--------------+------------+-----------+-----------+
|     city|employee_count|total_salary| avg_salary| max_salary|
+---------+--------------+------------+-----------+-----------+
|    Delhi|             1|       92000|    92000.0|      92000|
|Hyderabad|             1|       78000|    78000.0|      78000|
|   Mumbai|             2|      180000|    90000.0|      95000|
|     Pune|             1|       67000|    67000.0|      67000|
+---------+--------------+------------+-----------+-----------+
```

8.2 Window Functions — Advanced Analytics

Window functions perform calculations ACROSS rows related to the current row, without collapsing them into groups. This is the most interview-tested advanced PySpark topic.

Window Functions — rank, row_number, lag, lead, running sum

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Define a window specification
w_city = Window.partitionBy('city').orderBy(F.col('salary').desc())

# row_number() — unique sequential number per partition
df = df.withColumn('row_num', F.row_number().over(w_city))

# rank() — same rank for ties, gaps in sequence
df = df.withColumn('rank', F.rank().over(w_city))

# dense_rank() — same rank for ties, NO gaps
df = df.withColumn('dense_rank', F.dense_rank().over(w_city))

# lag() — previous row's value
df = df.withColumn('prev_salary',
    F.lag('salary', 1).over(w_city))

# lead() — next row's value
df = df.withColumn('next_salary',
    F.lead('salary', 1).over(w_city))

# Running total (cumulative sum)
w_cumsum = Window.partitionBy('dept') \
    .orderBy('salary') \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df = df.withColumn('running_total', F.sum('salary').over(w_cumsum))

# Percent rank
df = df.withColumn('pct_rank', F.percent_rank().over(w_city))

# ntile() — divide into N equal buckets
df = df.withColumn('quartile', F.ntile(4).over(w_city))
```

```
# Top N per group (most common interview pattern!)
top2_per_city = df.filter(F.col('row_num') <= 2)
top2_per_city.show()
```

Module 9: Joins — Combining DataFrames

Joins are the most complex and most interview-tested topic in PySpark. Every data engineer must master all join types, know which triggers a shuffle, and understand the broadcast join optimization.

> **Sateesh Says 💡**
>
> *Joins are like a marriage between two tables. Sometimes both parties match (inner join). Sometimes one side has extras (outer join).*
>
> *In PySpark, every join is potentially a SHUFFLE unless you use broadcast(). That shuffle is the #1 performance bottleneck.*
>
> *HDFC Bank joining 500 million transactions with a 10,000-row branch lookup table? Use broadcast join on the lookup table — saves MASSIVE network traffic!*

9.1 All Join Types

All PySpark join types — complete reference

```
# Setup: two DataFrames
emp_df = spark.createDataFrame([
    (1,'Raj','Mumbai',10),
    (2,'Priya','Delhi',20),
    (3,'Amit','Hyd',10),
    (4,'Sneha','Mumbai',30),   # dept 30 doesn't exist in dept_df
], ['emp_id','name','city','dept_id'])

dept_df = spark.createDataFrame([
    (10,'Analytics'),
    (20,'DataEng'),
    (40,'Marketing'),   # no employee in dept 40
], ['dept_id','dept_name'])

# INNER JOIN — only matching rows from both sides
emp_df.join(dept_df, on='dept_id', how='inner').show()
# Result: Raj, Priya, Amit (Sneha excluded — dept 30 not in dept_df)

# LEFT JOIN — all left rows + matching right rows
emp_df.join(dept_df, on='dept_id', how='left').show()
# Result: All 4 employees. Sneha has null dept_name

# RIGHT JOIN — all right rows + matching left rows
emp_df.join(dept_df, on='dept_id', how='right').show()
# Result: Analytics(Raj,Amit), DataEng(Priya), Marketing(null)

# FULL OUTER JOIN — all rows from both sides
emp_df.join(dept_df, on='dept_id', how='full').show()
emp_df.join(dept_df, on='dept_id', how='outer').show()  # same

# CROSS JOIN — every row from left x every row from right (Cartesian)
emp_df.crossJoin(dept_df).show()   # 4 x 3 = 12 rows! DANGEROUS!
```

```
# SEMI JOIN — keep left rows that HAVE a match on right
emp_df.join(dept_df, on='dept_id', how='semi').show()
# Result: Raj, Priya, Amit only (no dept columns included!)

# ANTI JOIN — keep left rows that DO NOT have a match on right
emp_df.join(dept_df, on='dept_id', how='anti').show()
# Result: Sneha only (dept 30 not in dept_df)

# JOIN ON MULTIPLE COLUMNS
df1.join(df2, on=['city','dept_id'], how='inner').show()

# JOIN WITH DIFFERENT COLUMN NAMES
df1.join(df2, df1.emp_dept == df2.dept_code, how='inner').show()
```

9.2 Broadcast Join — The Performance Game Changer

Broadcast Join — eliminate shuffle entirely

```
from pyspark.sql.functions import broadcast

# Problem: Joining 500 million orders with 50,000 product lookup
# Normal join = SHUFFLE 500M rows. VERY SLOW.

# SOLUTION: Broadcast the small lookup table to all Executors
result = orders_df.join(
    broadcast(products_df),   # <-- this sends products to all nodes
    on='product_id',
    how='inner'
)
# Now every Executor has a local copy of products_df
# No data movement for orders_df! Zero shuffle!

# Auto-broadcast threshold (Spark does this automatically below):
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', 10*1024*1024)
# Default 10MB. If a table is < 10MB, Spark broadcasts it auto.
# Set to -1 to disable auto-broadcast.
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', -1)
```

3 Types of Physical Join Strategies — Interview Gold!

1. Broadcast Hash Join (BHJ): Small table broadcast to all nodes. No shuffle. FASTEST. Use when one side < 10MB.
2. Sort Merge Join (SMJ): Both sides sorted on join key, then merged. Shuffle required. Default for large tables.
3. Shuffle Hash Join (SHJ): One side hashed and broadcast per partition. Used when one side is small-ish.

You can check which strategy Spark chose: df.join(df2,'id').explain() — look for 'BroadcastHashJoin' or 'SortMergeJoin' in the physical plan.

Module 10: Null Handling — Production-Critical Skills

NULL values are the #1 source of bugs in data pipelines. Understanding how PySpark handles nulls — and how to properly deal with them — is a MUST for senior data engineers.

> **Sateesh Says 💡**
>
> *In real data from Zomato, HDFC, or any enterprise, 10-30% of fields are null. Always.*
>
> *A data engineer who doesn't handle nulls properly causes wrong aggregations, broken joins, and angry business users.*
>
> *NULL in PySpark is like a ghost — it doesn't equal anything. NULL == NULL is FALSE! Handle it explicitly with isNull().*

10.1 Null Detection and Counting

Detecting and counting nulls in every column

```
from pyspark.sql import functions as F

# Count nulls in each column
null_counts = df.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in df.columns
])
null_counts.show()

# Null percentage per column
total = df.count()
null_pct = df.select([
    (F.count(F.when(F.col(c).isNull(), c)) / total * 100)
    .alias(f'{c}_null_pct')
    for c in df.columns
])
null_pct.show()

# Filter rows WHERE a column IS null
df.filter(F.col('salary').isNull()).show()
df.filter("salary IS NULL").show()     # SQL style

# Filter rows WHERE column is NOT null
df.filter(F.col('salary').isNotNull()).show()
```

10.2 dropna() — Remove Rows with Nulls

dropna() — All options

```
# Drop rows where ANY column is null (strictest)
df.dropna().show()
df.dropna(how='any').show()   # same as above

# Drop rows where ALL columns are null
df.dropna(how='all').show()

# Drop rows where specific columns have nulls
df.dropna(subset=['salary', 'city']).show()

# Drop rows where fewer than N non-null values exist
df.dropna(thresh=3).show()    # keep rows with >= 3 non-null cols
```

10.3 fillna() — Replace Nulls with Values

fillna() / na.fill() — All patterns

```
# Fill ALL columns with the same value (type-safe replacement)
df.fillna(0).show()          # fill all numeric nulls with 0
df.fillna('Unknown').show()  # fill all string nulls with 'Unknown'
df.fillna(False).show()      # fill all boolean nulls

# Fill specific columns with specific values (MOST USEFUL!)
df.fillna({
    'salary':    0,
    'city':      'Unknown',
    'dept':      'Unassigned',
    'join_date': '2024-01-01'
}).show()

# Fill with computed value (e.g., mean salary)
mean_salary = df.select(F.avg('salary')).collect()[0][0]
df = df.fillna({'salary': mean_salary})

# Using na.fill() — same as fillna()
df.na.fill(0).show()
df.na.fill({'salary':0,'city':'Unknown'}).show()

# Replace specific values (not just nulls)
df.na.replace(['Hyd','Pune'], ['Hyderabad','Pune, MH']).show()
```

10.4 Null Behavior in Operations — CRITICAL!

How nulls affect groupBy, sort, and join

```
# Null in arithmetic: any null = result null
df.withColumn('bonus', F.col('salary') * 0.1).show()
# If salary is null -> bonus is null

# Safe arithmetic with coalesce() or nvl()
df.withColumn('bonus',
    F.coalesce(F.col('salary'), F.lit(0)) * 0.1).show()
```

```
# Null in groupBy: NULL forms its own group!
df.groupBy('city').count().show()    # nulls appear as 'null' group

# Null in sort: nulls appear at end by default
df.orderBy(F.col('salary').asc_nulls_last()).show()
df.orderBy(F.col('salary').desc_nulls_first()).show()

# Null in join: null keys DO NOT match (null != null in joins)
# Rows with null join key are excluded from inner join result!
# Use left join + isNull filter to find unmatched records.

# NULL-safe equality comparison (===)
df1.join(df2, df1.city.eqNullSafe(df2.city), 'inner').show()
# null == null returns TRUE with eqNullSafe (unlike normal ==)
```

Module 11: union(), unionAll() & set operations

11.1 union() and unionAll()

Combining DataFrames vertically

```
df_mumbai = spark.createDataFrame([
    ('Raj','Mumbai',85000),('Sneha','Mumbai',95000)
], ['name','city','salary'])

df_delhi = spark.createDataFrame([
    ('Priya','Delhi',92000),('Karan','Delhi',71000)
], ['name','city','salary'])

# union() — combines rows, KEEPS duplicates, matches by POSITION
combined = df_mumbai.union(df_delhi)
combined.show()  # 4 rows

# unionAll() — IDENTICAL to union() in Spark 2.x+
# (in Spark 1.x unionAll kept dups, union removed them — NOT now!)
combined = df_mumbai.unionAll(df_delhi)  # same result

# unionByName() — matches by COLUMN NAME not position!
# SAFER when columns might be in different order
df1 = spark.createDataFrame([(1,'Raj')],['id','name'])
df2 = spark.createDataFrame([('Priya',2)],['name','id'])
df1.unionByName(df2).show()    # correctly aligns id and name
# df1.union(df2) would swap id and name! Bug!

# Remove duplicates after union
combined.distinct().show()

# Union multiple DataFrames
from functools import reduce
dfs = [df1, df2, df3, df4]
result = reduce(lambda a,b: a.unionByName(b), dfs)
```

Common Mistake / Watch Out!

union() matches columns by POSITION, not by name. If df1 has ['id','name'] and df2 has ['name','id'], union() will produce wrong results — id values mixed into name column!

ALWAYS use unionByName() when combining DataFrames that may have different column ordering.

union() in PySpark 2.x+ does NOT remove duplicates — use .distinct() after if deduplication is needed.

Module 12: Writing DataFrames — Output Formats & Modes

Reading data in and reading data out are both core skills. Understanding write modes, partition strategies, and file format options is critical for production pipelines.

12.1 Write Modes

Writing DataFrames — all modes and formats

```
# Write MODES — what to do if the output already exists
df.write.mode('overwrite').csv('output/')   # Delete and recreate
df.write.mode('append').csv('output/')      # Add to existing
df.write.mode('ignore').csv('output/')      # Skip if exists
df.write.mode('error').csv('output/')       # Throw error if exists
# Default mode = 'error'

# Write as different formats
df.write.mode('overwrite').csv('output/csv/')
df.write.mode('overwrite').json('output/json/')
df.write.mode('overwrite').parquet('output/parquet/')
df.write.mode('overwrite').orc('output/orc/')
df.write.mode('overwrite').format('delta').save('output/delta/')

# Write with options
df.write \
  .mode('overwrite') \
  .option('header','true') \          # for CSV: include header
  .option('compression','gzip') \     # compress output files
  .option('sep', '|') \              # custom delimiter for CSV
  .csv('output/csv/')

# Write partitioned output (creates folder structure by column)
df.write \
  .mode('overwrite') \
  .partitionBy('city', 'dept') \
  .parquet('output/emp/')
# Creates: output/emp/city=Mumbai/dept=Analytics/part-0000.parquet

# Control number of output files (important for small file problem!)
df.repartition(1).write.mode('overwrite').csv('output/single/')
df.coalesce(5).write.mode('overwrite').parquet('output/5files/')

# Write to Hive table / Delta table
df.write.mode('overwrite').saveAsTable('employees')
df.write.mode('overwrite').saveAsTable('catalog.schema.employees')
```

Module 13: Interview Master Sheet — 25 Questions

Sateesh Says 💡

These 25 questions are compiled from real PySpark interviews at Amazon, Flipkart, Walmart Global Tech, TCS Digital, Infosys Specialist, Wipro, Capgemini, and Databricks certification exams.

Study the REASONING behind each answer — not just the answer itself. Interviewers ask follow-up questions!

13.1 DataFrame Fundamentals

IQ-01 **What is a DataFrame in PySpark? How is it different from a Pandas DataFrame?**

A PySpark DataFrame is a distributed, immutable table with named typed columns, split into partitions across cluster nodes. A Pandas DataFrame lives entirely in a single machine's RAM. PySpark DataFrames can handle petabytes; Pandas is limited to single-node memory (~10-50 GB practically).

IQ-02 **What is the difference between show(), collect(), and toPandas()?**

show() prints N rows to the console (safe on any size). collect() returns ALL rows as a Python list to the Driver (dangerous on large data — causes OOM). toPandas() converts entire DataFrame to Pandas (also dangerous). Only use collect()/toPandas() after heavy aggregation reduces data to manageable size.

IQ-03 **Why should you always provide an explicit schema when reading CSV?**

inferSchema=True reads the file TWICE — once to detect types, once to load. On large files this doubles read time. Also, type inference can be wrong: a column of '001','002' gets inferred as Integer, losing leading zeros. Explicit schema is faster and deterministic.

IQ-04 **What is the difference between filter() and where()?**

They are IDENTICAL — where() is just an alias for filter(). Both are lazy transformations. Use whichever reads more naturally for your team. SQL-background developers often prefer where().

IQ-05 **Why does df.filter(df.city=='Mumbai' and df.salary>80000) produce wrong results?**

Python's 'and' operator evaluates the boolean of the Column object (which is always True for non-null Column), not the logical combination. You must use the '&' operator with parentheses: (df.city=='Mumbai') & (df.salary>80000). This is a classic Python-vs-PySpark operator confusion bug.

IQ-06 What is the difference between distinct() and dropDuplicates(subset)?

distinct() removes rows where ALL columns are identical. dropDuplicates(['col1','col2']) removes rows where specified columns form a duplicate, keeping other column values from the first occurrence. Both trigger a shuffle internally.

IQ-07 Explain withColumn() — can it update an existing column?

Yes! If you pass an existing column name as the first argument, withColumn() REPLACES that column with the new expression. Example: df.withColumn('salary', col('salary').cast(DoubleType())) converts the salary column to Double in place.

IQ-08 What is selectExpr() and when would you use it over select()?

selectExpr() accepts SQL expression strings directly. Use it when the expression is complex and would be verbose with the column API — CASE WHEN, regex, window functions, date arithmetic. Both compile to the same execution plan, so there's no performance difference.

IQ-09 What is the difference between union() and unionByName()?

union() stacks rows matching by COLUMN POSITION — if column order differs between DataFrames, values get misaligned (bug!). unionByName() matches by COLUMN NAME — safe regardless of column ordering. Always prefer unionByName() in production.

IQ-10 What is the small file problem and how do you solve it?

When a DataFrame has many small partitions, writing produces many small files (e.g., 10,000 x 1MB files). Reading many small files is slow due to filesystem overhead. Fix: df.coalesce(N).write... to merge partitions before writing. Alternatively, use Delta Lake OPTIMIZE command which compacts files automatically.

IQ-11 Explain all 4 write modes in PySpark.

overwrite: deletes existing data and writes fresh. append: adds new data to existing. ignore: skips write if path already exists (no error). error (default): throws AnalysisException if path already exists. In production ETL, overwrite is used for full loads; append for incremental.

IQ-12 What is the difference between sort()/orderBy() and sortWithinPartitions()?

sort()/orderBy() does a GLOBAL sort — produces a globally ordered result with a shuffle. Very expensive at scale. sortWithinPartitions() sorts WITHIN each partition independently — no shuffle, much faster. Use sortWithinPartitions() when writing sorted Parquet files but don't need global order.

IQ-13 How do you count null values in each column of a DataFrame?

Use: df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns]).show(). This creates one count column per original column using list comprehension — a common

one-liner tested in interviews.

IQ-14 What is the difference between fillna() and na.replace()?

fillna() replaces NULL values with a specified value. na.replace() replaces specific non-null values with other values — like replacing 'Hyd' with 'Hyderabad' across a column. Both use the na namespace: df.na.fill() and df.na.replace() are equivalent to df.fillna() and df.replace().

IQ-15 How does null behave in a join?

NULL keys do NOT match in joins by default (null != null in SQL semantics). Rows with null join key are excluded from inner join results. For null-safe joins use eqNullSafe() or coalesce the join key to a sentinel value before joining. This is a common source of unexpected data loss in pipelines!

IQ-16 What is a Broadcast Join? When should you use it?

A broadcast join sends a copy of the smaller DataFrame to every Executor, so the join happens locally without network shuffle. Use when one DataFrame is smaller than spark.sql.autoBroadcastJoinThreshold (default 10MB). Force it with broadcast(df). Eliminates the shuffle entirely — often 10-50x faster for lookups.

IQ-17 Explain all 7 join types in PySpark.

inner: only matching rows. left/left_outer: all left rows, matching right (nulls where no match). right/right_outer: all right rows, matching left (nulls). full/outer: all rows from both sides. semi/left_semi: left rows that HAVE a match (no right columns). anti/left_anti: left rows that DO NOT have a match. cross: Cartesian product (dangerous — N*M rows).

IQ-18 What is the difference between groupBy().count() and count()?

df.groupBy('city').count() groups by city and counts rows per group — returns a DataFrame. df.count() counts TOTAL rows in the DataFrame — returns a single integer. groupBy().count() is a transformation+action; count() alone is an action.

IQ-19 How do you find the top N rows per group?

Use row_number() window function: Window.partitionBy('group_col').orderBy(col('sort_col').desc()). Add row_num column, then filter(col('row_num') <= N). This is one of the most common interview coding questions.

IQ-20 What is the difference between rank(), dense_rank(), and row_number()?

row_number(): always unique, sequential (1,2,3,4). rank(): ties get same number, then skips (1,1,3,4 — gap). dense_rank(): ties get same number, no gap (1,1,2,3). Memorize with example: salaries 100,100,80,70 -> row_number=1,2,3,4; rank=1,1,3,4; dense_rank=1,1,2,3.

IQ-21 What is collect_list() vs collect_set() in groupBy?

collect_list() aggregates all values into an array, PRESERVING duplicates and order. collect_set() aggregates into an array of UNIQUE values only (no duplicates, no guaranteed order). Example: df.groupBy('dept').agg(collect_list('city'), collect_set('city')).

IQ-22 What is the pivot() function and when do you use it?

pivot() transforms unique values of one column into multiple columns in the output — like SQL PIVOT or Excel pivot tables. Example: df.groupBy('dept').pivot('city').agg(avg('salary')) creates one column per city. Specify values explicitly in pivot('city',['Mumbai','Delhi']) for better performance — avoids an extra scan to find distinct values.

IQ-23 What is the difference between .cache() and .persist() with a DataFrame?

cache() = persist(StorageLevel.MEMORY_AND_DISK) — stores in memory, falls back to disk. persist() lets you choose the storage level: MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_2 (replicated). Use cache() for convenience; use persist(DISK_ONLY) when the DataFrame is too large for memory.

IQ-24 How do you add an auto-increment ID column (like a sequence number) in PySpark?

Use monotonically_increasing_id() — it generates unique but NOT consecutive IDs: df.withColumn('id', F.monotonically_increasing_id()). For consecutive IDs use zipWithIndex() on the underlying RDD or row_number() over a window with no partition. Note: monotonically_increasing_id() is NOT consistent across reruns — don't use as a primary key.

IQ-25 What does explain() do and how do you use it to diagnose performance?

df.explain() prints the physical execution plan. df.explain(True) prints logical + physical plans. Look for: BroadcastHashJoin (fast) vs SortMergeJoin (slow), Exchange (shuffle operations), InMemoryRelation (cache hit), FileScan with PushedFilters (predicate pushdown working). Senior engineers read explain() output to diagnose every slow query.

Module 14: Quick Reference — All Methods at a Glance

Method	Category	Quick Syntax	Returns
createDataFrame()	Create	spark.createDataFrame(data, schema)	DataFrame
read.csv()	Create	spark.read.option(...).csv(path)	DataFrame
read.parquet()	Create	spark.read.parquet(path)	DataFrame
read.json()	Create	spark.read.json(path)	DataFrame
printSchema()	Inspect	df.printSchema()	None (prints)
show()	Inspect	df.show(n, truncate=False)	None (prints)
describe()	Inspect	df.describe('col').show()	DataFrame
count()	Inspect	df.count()	Integer (Action)
columns	Inspect	df.columns	List[str]
dtypes	Inspect	df.dtypes	List[Tuple]
first()	Inspect	df.first()	Row
collect()	Inspect	df.collect()	List[Row] ⚠️
select()	Transform	df.select('c1','c2')	DataFrame
selectExpr()	Transform	df.selectExpr('c1*2 as c1x2')	DataFrame
filter()/where()	Transform	df.filter(col('c')=='v')	DataFrame
withColumn()	Transform	df.withColumn('c',expr)	DataFrame
withColumnRenamed()	Transform	df.withColumnRenamed('old','new')	DataFrame
drop()	Transform	df.drop('col1','col2')	DataFrame
orderBy()/sort()	Transform	df.orderBy(col('x').desc())	DataFrame
distinct()	Transform	df.distinct()	DataFrame
dropDuplicates()	Transform	df.dropDuplicates(['c1','c2'])	DataFrame
groupBy().agg()	Aggregate	df.groupBy('c').agg(sum('s'))	DataFrame

groupBy().count()	Aggregate	df.groupBy('c').count()	DataFrame
join()	Join	df1.join(df2,on='id',how='inner')	DataFrame
union()	Combine	df1.union(df2)	DataFrame
unionByName()	Combine	df1.unionByName(df2)	DataFrame
fillna()	Nulls	df.fillna({'col':0})	DataFrame
dropna()	Nulls	df.dropna(subset=['c1'])	DataFrame
cache()	Perf	df.cache()	DataFrame
persist()	Perf	df.persist(StorageLevel.DISK_ONLY)	DataFrame
repartition()	Perf	df.repartition(200,'city')	DataFrame
coalesce()	Perf	df.coalesce(10)	DataFrame
write.parquet()	Write	df.write.mode('ow').parquet(p)	None (Action)
write.csv()	Write	df.write.mode('ow').csv(path)	None (Action)
explain()	Debug	df.explain(True)	None (prints)

Sateesh Says 💡

You have just studied the complete PySpark DataFrame 01 expert notes.

STEP 1: Re-read the code blocks and type every example yourself — never copy-paste when learning.

STEP 2: For each concept, ask yourself: 'What happens internally? What triggers a shuffle? What is lazy?'

STEP 3: Attempt all 25 interview questions without looking at the answers. Then verify.

STEP 4: Practice on real data. Download the Zomato orders dataset or IPL match data from Kaggle — apply every transformation here.

STEP 5: Open Spark UI (localhost:4040) and look at the DAG for your groupBy jobs. See the shuffle.

Remember: The best PySpark engineers are not the ones who memorised the API — they are the ones who understand WHY.

All the best! — Sateesh Pabbathi, MCT | www.levelupedu.net